

INTERVIEW PREP • Q&A DRILLS • PYTHON EXAMPLES

Machine Learning Pocket Notes Interview

**Q&A: 200+ Questions, Practical Python Examples,
Chapter-by-Chapter Overview and Full Q&A Drill.**

205 deep-dive interview questions across 17 chapters, plus 204 rapid-fire Q&A drills across 14 topics---from supervised learning fundamentals to Transformers, MLOps, and explainable AI.



Contents

1 Foundations of Machine Learning	1
1 — What is the difference between supervised, unsupervised, and reinforcement learning?	1
2 — Explain the bias–variance trade-off. How does it guide model selection?	1
3 — What is the curse of dimensionality and how do you address it?	2
2 Regression	3
4 — Derive the ordinary least-squares estimator and state when it is BLUE.	3
5 — Compare Ridge, Lasso, and Elastic Net regularisation. When do you use each?	3
3 Classification	4
6 — Explain logistic regression: the model, training objective, and its probabilistic interpretation.	4
7 — When would you choose Random Forest over XGBoost, or vice versa?	5
8 — Explain precision, recall, F1, and AUC-ROC. How do class imbalance and decision threshold affect them?	5
4 Decision Trees and Ensemble Methods	6
9 — How does a decision tree choose which feature and split point to use?	6
10 — Explain gradient boosting from first principles. What problem does it solve that AdaBoost does not?	7
5 Support Vector Machines	7
11 — Derive the SVM primal and dual. What is the kernel trick?	7
6 Neural Networks and Deep Learning	8
12 — Explain backpropagation. Derive gradients for a two-layer network.	8
13 — Compare Adam, SGD with momentum, and RMSProp. When should you prefer each?	9
7 Convolutional Neural Networks	10
14 — How does a convolution operation work, and why is parameter sharing important?	10
8 Recurrent Networks and Sequence Models	10
15 — Why do vanilla RNNs fail on long sequences, and how do LSTMs fix this?	10
9 Transformers and Attention	11
16 — Derive scaled dot-product attention. Why the scaling factor $\sqrt{d_k}$?	11
10 Unsupervised Learning and Dimensionality Reduction	12
17 — Explain the k-means algorithm. What are its limitations and how does k-means++ address initialisation?	12
11 Feature Engineering and Preprocessing	12
18 — How do you handle missing data? Compare deletion, imputation, and model-based approaches.	13
12 Model Evaluation and Selection	13
19 — Explain k-fold cross-validation. When would you use stratified, grouped, or time-series CV?	13
13 Bayesian Methods	14
20 — Explain Bayesian inference. What is the trade-off between Bayesian and frequentist approaches?	14
14 Natural Language Processing	15
21 — Explain word embeddings: Word2Vec, GloVe, and contextual embeddings (BERT). What are their key differences?	15
15 Reinforcement Learning	15
22 — Explain the Bellman equation and Q-learning. Why is experience replay and a target network needed in DQN?	15
16 MLOps and Production ML	16
23 — What is data drift and concept drift? How do you detect and respond to each?	16
24 — Describe a production ML serving architecture. How do you handle latency, throughput, and reliability?	17
17 Explainability and Ethics in ML	18
25 — Explain SHAP values. Why are they theoretically grounded compared to other feature importance methods?	18

— Q&A Drill: 200+ Interview Questions —	19
18 Q&A — Foundations & Learning Paradigms	19
Q1 — What is supervised learning?	19
Q2 — What is unsupervised learning?	19
Q3 — What is overfitting, and how can you prevent it?	20
Q11 — What is regularisation in machine learning?	20
Q12 — What is the bias–variance tradeoff?	20
Q15 — What is feature selection?	20
Q17 — How does Random Forest work?	20
Q18 — What is Support Vector Machine (SVM)?	21
Q21 — What are neural networks?	21
Q23 — What is dropout in neural networks?	21
Q29 — What is early stopping?	21
Q40 — What is pruning in decision trees?	21
Q52 — Give an example of transfer learning.	21
Q53 — What is an autoencoder?	22
Q74 — What is early stopping in neural networks? (revisited)	22
Q85 — What is dropout in neural networks? (revisited)	22
Q110 — Difference between supervised and unsupervised learning?	22
Q111 — What is feature drift?	22
Q112 — What is concept drift?	22
Q113 — What is semi-supervised learning?	22
Q114 — What is active learning?	22
Q118 — What is label smoothing?	22
Q135 — What is XGBoost and why is it popular?	22
Q136 — What is regularisation (L1, L2) and how do they help?	23
Q137 — How does early stopping improve generalisation?	23
Q144 — What is transfer learning? (Advanced)	23
Q147 — What is label propagation in semi-supervised learning?	23
Q148 — What are variational autoencoders (VAEs)?	23
Q151 — What is transfer learning domain adaptation?	23
Q157 — Bagging vs. boosting in real-world use?	24
Q163 — What is self-supervised learning?	24
Q185 — What is active sampling in ML?	24
Q186 — What is knowledge transfer in multi-task learning?	24
Q188 — What is elastic net regularisation?	24
Q190 — What is polynomial regression?	24
Q191 — What are the limitations of deep learning?	24
Q192 — What is active learning and when is it useful?	25
Q194 — What is batch normalisation and why is it critical?	25
Q196 — What is dropout and how does it prevent overfitting?	25
Q197 — What is early stopping? (concise)	25
Q201 — Why is transfer learning transformative for small datasets?	25
Q203 — What is feature drift and concept drift?	25
19 Q&A — Data Preparation & Feature Engineering	26
Q6 — What is feature scaling, and why is it important?	26
Q13 — Explain one-hot encoding.	26
Q25 — What is data normalisation?	26

Q26 — How do you handle missing values in data?	26
Q41 — What is feature engineering?	26
Q42 — Give examples of feature engineering.	26
Q43 — What is label encoding?	27
Q59 — What is DBSCAN?	27
Q76 — What is batch normalisation?	27
Q84 — What is data augmentation?	27
Q106 — What is an outlier?	27
Q121 — What is a data pipeline?	27
Q123 — What is data versioning and why is it important?	27
Q129 — What is feature hashing?	27
Q132 — What is cross-validation leakage and how do you avoid it?	28
Q140 — How do you handle categorical variables in tree-based models?	28
Q142 — What are positional encodings in Transformer models?	28
Q153 — What is CatBoost and how does it handle categorical features?	28
Q160 — What are Bayesian neural networks, and their advantage?	28
Q167 — What is the Gumbel-softmax trick?	28
Q176 — What is robust regression?	28
Q193 — What is the role of feature importance in ML?	28
20 Q&A — Model Evaluation, Validation & Experimentation	29
Q7 — How do you evaluate a classification model?	29
Q8 — Explain cross-validation.	29
Q9 — What is hyperparameter tuning?	29
Q10 — What is a confusion matrix?	29
Q19 — What is the purpose of the ROC curve?	29
Q20 — What is precision and recall?	30
Q30 — What is model deployment?	30
Q31 — What is ensemble learning?	30
Q36 — Explain gradient boosting.	30
Q47 — What are confusion matrix terms: TP, FP, TN, FN?	30
Q48 — What is the F1 score?	30
Q54 — What are convolutional neural networks (CNNs)?	30
Q56 — What is a confusion matrix used for?	30
Q60 — What is silhouette score?	31
Q62 — What is a ROC-AUC score?	31
Q66 — What is the precision-recall tradeoff?	31
Q90 — What is a Pipeline in scikit-learn?	31
Q92 — What is the Transformer architecture?	31
Q98 — What is reinforcement learning?	31
Q100 — What is a Markov Decision Process (MDP)?	31
Q103 — What is data leakage?	31
Q109 — What is hyperparameter optimisation?	31
Q116 — What is a precision-recall curve?	32
Q117 — What is model calibration?	32
Q124 — What is a hyperparameter search space?	32
Q133 — What is stratified k-fold cross-validation?	32
Q134 — Explain the ROC curve, AUC, and their practical significance.	32
Q139 — Difference between recall and specificity?	32

Q149 — What is quantisation in deploying ML models?	32
Q156 — What is out-of-bag (OOB) error in random forests?	33
Q158 — What is pipelining in MLOps?	33
Q168 — What is data sharding?	33
Q177 — What is evolutionary computation in ML?	33
Q178 — What is multi-objective optimisation?	33
Q179 — What is transferability in adversarial ML?	33
Q180 — What is hyperparameter tuning with Bayesian Optimisation?	33
Q195 — Why are ensemble models used in Kaggle competitions?	33
Q199 — What is class imbalance and its impact?	33
Q200 — How do you address class imbalance?	34
Q204 — What is model calibration and why is it important?	34
Q205 — What is bootstrapping and why is it used?	34
21 Q&A — Classical Supervised Algorithms	34
Q4 — Difference between classification and regression?	34
Q5 — How does k-Nearest Neighbours (kNN) work?	34
Q22 — What is a perceptron?	35
Q27 — What is cross entropy loss?	35
Q37 — What are decision trees and how do they work?	35
Q39 — What is entropy in decision trees?	35
Q44 — Parametric vs. non-parametric models?	35
Q69 — What is the bias node in neural networks?	35
Q71 — What is the softmax function?	35
Q95 — What is multi-class classification?	36
Q96 — What is multi-label classification?	36
Q107 — What is time series forecasting?	36
Q108 — What is ARIMA?	36
Q119 — Hard vs. soft classification?	36
Q122 — What is multi-output regression?	36
Q125 — What is quantile regression?	36
Q126 — What is distance metric learning?	36
Q131 — What is ensemble stacking?	36
Q154 — What is LIME?	37
Q189 — What is intercept bias in regression?	37
22 Q&A — Ensembles & Boosting	37
Q16 — Difference between bagging and boosting?	37
Q32 — Name common ensemble algorithms.	37
Q33 — What is stacking in ensembles?	37
Q34 — What is bagging and give an example?	38
Q35 — What is boosting and give an example?	38
Q50 — How can you handle imbalanced datasets?	38
Q89 — Ensemble bagging vs. boosting in one line?	38
23 Q&A — Unsupervised Learning & Clustering	38
Q57 — What is k-means clustering?	38
Q58 — What is hierarchical clustering?	38
Q67 — What is anomaly detection?	39
Q70 — What is the elbow method in clustering?	39

Q127 — What is the silhouette coefficient formula?	39
24 Q&A — Dimensionality Reduction	39
Q14 — What is Principal Component Analysis (PCA)?	39
Q63 — Explain dimensionality reduction.	39
Q64 — What is t-SNE?	39
Q175 — What is singular value decomposition (SVD)?	39
25 Q&A — Deep Learning Core Concepts	40
Q24 — Explain batch gradient descent.	40
Q28 — What does the learning rate control?	40
Q55 — What are recurrent neural networks (RNNs)?	40
Q72 — What is the activation function in a neural network?	40
Q73 — What is the ReLU activation function?	40
Q75 — What is Xavier/Glorot initialisation?	40
Q77 — What is gradient vanishing/explosion?	40
Q78 — What is an epoch?	40
Q79 — What is mini-batch gradient descent?	40
Q80 — What is the main role of the loss function?	40
Q81 — What is a generator in deep learning?	40
Q82 — What are Generative Adversarial Networks (GANs)?	41
Q83 — What is LSTM?	41
Q88 — What is a learning rate scheduler?	41
Q91 — What is the attention mechanism in deep learning?	41
Q101 — What is the exploding gradient problem?	41
Q102 — What is a custom loss function?	41
Q128 — What mainly causes the vanishing gradient problem?	41
Q138 — Explain learning rate annealing and why to use it.	41
Q145 — Bag-of-words vs. embeddings for NLP?	41
Q173 — What is a graph neural network (GNN)?	42
Q181 — What is Nash Equilibrium in ML?	42
Q182 — What is hierarchical reinforcement learning?	42
26 Q&A — NLP & Transformers	42
Q68 — What are word embeddings?	42
Q86 — What is Word2Vec?	42
Q87 — Bag of Words vs. TF-IDF?	42
Q93 — What is BERT?	42
Q130 — What is negative sampling?	43
Q141 — What is multi-head attention in Transformers?	43
Q143 — What is attention masking?	43
27 Q&A — Reinforcement Learning & Decision Making	43
Q99 — What is Q-learning?	43
Q164 — What is reinforcement learning reward shaping?	43
Q165 — What is Monte Carlo Tree Search?	43
Q166 — What is bandit learning?	43
Q183 — Exploration vs. exploitation in RL?	43
28 Q&A — MLOps, Deployment & Production	44
Q61 — What is model serialisation and why is it important?	44
Q65 — Name tools for model deployment.	44

Q198 — Why is interpretability increasingly important in ML?	44
29 Q&A — Explainability, Robustness & Privacy	44
Q45 — What is model interpretability and why is it important?	44
Q46 — List model interpretability techniques.	44
Q104 — What is explainable AI (XAI)?	44
Q105 — What is feature importance?	44
Q155 — What is SHAP and how does it differ from LIME?	45
Q159 — Explainability vs. interpretability?	45
Q161 — What is adversarial training in deep learning?	45
Q169 — What is federated learning?	45
Q170 — What is privacy-preserving ML?	45
Q171 — What is differential privacy?	45
Q202 — What are SHAP values and how do they aid explainability?	45
30 Q&A — Graphs & Advanced Topics	46
Q150 — Explain knowledge distillation.	46
Q152 — What is meta-learning (“learning to learn”)?	46
Q162 — What is curriculum learning?	46
Q172 — What is knowledge graph embedding?	46
Q174 — What is node2vec?	46
31 Q&A — Other Topics	46
Q38 — What is Gini impurity?	46
Q49 — What is class imbalance?	46
Q94 — What is fine-tuning in deep learning?	46
Q97 — What is zero-shot learning?	46
Q115 — What is SMOTE?	47
Q120 — Early fusion vs. late fusion in multimodal learning?	47
Q146 — What are residual connections and why do they work?	47
Q184 — What is the curse of dimensionality?	47
Q187 — What is continual learning?	47

1 Foundations of Machine Learning

Chapter Overview

★ Key Insight

Machine learning is the discipline of designing algorithms that improve automatically through experience. Rather than hand-coding rules, we expose a model to data and let it discover patterns. The three canonical learning paradigms — supervised, unsupervised, and reinforcement learning — differ in what signal drives that discovery.

Supervised learning maps inputs $\mathbf{x}$ to outputs y using labelled examples $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$. The learning algorithm minimises an empirical loss $\mathcal{L}(\theta) = \frac{1}{N} \sum_i \ell(f_\theta(\mathbf{x}_i), y_i)$ and the hope is that the learnt f_θ generalises to unseen inputs. **Unsupervised learning** finds structure — clusters, latent factors, density estimates — without labels. **Reinforcement learning** trains an agent to maximise cumulative reward through interaction with an environment.

The **bias–variance trade-off** is the central tension in supervised learning: simpler models underfit (high bias), complex models overfit (high variance). Regularisation, cross-validation, and ensemble methods are the primary tools for navigating it. The **No Free Lunch theorem** formalises the intuition that no single learner is universally best: every algorithm’s advantage on some problems is paid for by disadvantage on others.

Q1 — What is the difference between supervised, unsupervised, and reinforcement learning?

Q: Define each paradigm and give a concrete industry example of each.

Answer:

Supervised learning The model trains on labelled pairs $(\mathbf{x}, y)$ and learns a mapping $f : \mathbf{x} \rightarrow y$. The loss $\mathcal{L}$ measures prediction error and gradient descent adjusts parameters θ to minimise it. *Example:* fraud detection — each transaction is labelled *fraudulent* or *legitimate* and a gradient-boosted classifier learns the boundary.

Unsupervised learning No labels are provided. The algorithm discovers latent structure: clusters (k-means), manifolds (t-SNE/UMAP), generative distributions (VAEs, GANs), or low-rank factors (PCA, NMF). *Example:* customer segmentation — a retailer clusters purchase histories to identify distinct buyer personas without any pre-defined categories.

Reinforcement learning An agent observes state s_t , takes action a_t , and receives reward r_t from an environment. It learns a policy $\pi(a|s)$ that maximises expected cumulative discounted reward $\mathbb{E}[\sum_t \gamma^t r_t]$. *Example:* recommendation systems — a streaming platform trains an RL agent to select which video to surface next, optimising for long-term watch time rather than immediate clicks.

✓ Best Practice

In production, the boundaries blur: semi-supervised methods exploit cheap unlabelled data alongside scarce labels; self-supervised pre-training (BERT, GPT) uses unsupervised objectives on massive corpora then fine-tunes with supervision on small labelled sets. Always ask “*where does my signal come from?*” before selecting a paradigm.

Q2 — Explain the bias–variance trade-off. How does it guide model selection?

Q: Derive the bias–variance decomposition and explain its practical implications.

Answer:

For a regression target $y = f(\mathbf{x}) + \epsilon$ with $\mathbb{E}[\epsilon] = 0$, $\text{Var}[\epsilon] = \sigma^2$, the expected squared test error of estimator $\hat{f}$ at point $\mathbf{x}_0$ decomposes as:

$$\mathbb{E}[(y - \hat{f}(\mathbf{x}_0))^2] = \underbrace{\left(f(\mathbf{x}_0) - \mathbb{E}[\hat{f}(\mathbf{x}_0)]\right)^2}_{\text{Bias}^2} + \underbrace{\text{Var}[\hat{f}(\mathbf{x}_0)]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible noise}}$$

- **High bias** (underfitting): model is too simple to capture the true f . Error is high on *both* train and test sets. Fix: increase model capacity, add features, reduce regularisation.
- **High variance** (overfitting): model memorises training noise. Low train error, high test error. Fix: add regularisation (L1/L2, dropout), gather more data, reduce capacity, or use ensembles.
- **Sweet spot**: the right complexity minimises total generalisation error. Cross-validation on a held-out validation set estimates where this is without touching the test set.

! Pitfall / Warning

Deep neural networks appear to violate the classical trade-off in the *double descent* regime: as model size or training steps grow past interpolation threshold, test error can *decrease* again. This is an active research area; the classical bias–variance story still guides feature engineering and regularisation choices but should not be applied naively to very large networks.

Q3 — What is the curse of dimensionality and how do you address it?

Q: Explain why high-dimensional data is problematic and list mitigation strategies.

Answer:

In d dimensions, the volume of the unit hypercube grows as $1^d = 1$, but the fraction of data within distance ϵ of any point shrinks exponentially. Concretely, to cover 10% of the unit hypercube with a hypercube in $d = 10$ dimensions requires edge length $0.1^{1/10} \approx 0.79$ — almost the entire range. Consequences:

1. **Sparsity**: N training samples become increasingly sparse as d grows; the nearest neighbours of a query point are not meaningfully “near”.
2. **Distance concentration**: the ratio max / min of pairwise distances converges to 1, making nearest-neighbour search uninformative.
3. **Exponential sample complexity**: parametric density estimation requires exponentially more data to achieve a fixed error level.

Mitigations:

- **Dimensionality reduction**: PCA (linear), autoencoders (non-linear), UMAP for visualisation. Retain 95% of explained variance as a heuristic threshold.
- **Feature selection**: Lasso (L1), mutual information, recursive feature elimination. Remove redundant or noisy features.

- **Regularisation:** penalise model complexity so it does not exploit noise in sparse regions.
- **Domain knowledge:** engineer low-dimensional representations that capture semantic meaning (e.g. word embeddings, graph node features).

2 Regression

Chapter Overview

Regression models predict a continuous target. **Linear regression** is the workhorse: it is analytically tractable, interpretable, and the basis for understanding more complex models. In practice most tabular regression problems use gradient-boosted trees (XGBoost, LightGBM) or neural networks, but linear models remain important baselines and diagnostic tools. Key concerns are feature scaling, multicollinearity, heteroscedasticity, and the choice of regularisation.

Q4 — Derive the ordinary least-squares estimator and state when it is BLUE.

Q: Walk through the OLS derivation and the Gauss–Markov conditions.

Answer:

Minimise the residual sum of squares $\text{RSS}(\beta) = \|\mathbf{y} - \mathbf{X}\beta\|_2^2$:

$$\frac{\partial \text{RSS}}{\partial \beta} = -2\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\beta) = \mathbf{0} \Rightarrow \hat{\beta}_{\text{OLS}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Under the **Gauss–Markov conditions** — (i) $\mathbb{E}[\epsilon] = \mathbf{0}$, (ii) $\text{Cov}[\epsilon] = \sigma^2 \mathbf{I}$ (homoscedasticity and no autocorrelation), (iii) $\mathbf{X}$ has full column rank — $\hat{\beta}_{\text{OLS}}$ is the **Best Linear Unbiased Estimator** (BLUE); among all linear unbiased estimators it has the smallest variance.

! Pitfall / Warning

$\mathbf{X}^\top \mathbf{X}$ is singular (non-invertible) when features are perfectly collinear or when $p > n$. Solutions: remove duplicate features, apply Ridge regression $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})$, or use the Moore–Penrose pseudoinverse.

Q5 — Compare Ridge, Lasso, and Elastic Net regularisation. When do you use each?

Q: Explain the penalties, their geometric intuition, and practical selection criteria.

Answer:

All three add a penalty to the OLS objective to shrink coefficients and reduce variance:

$$\hat{\beta} = \arg \min_{\beta} \underbrace{\|\mathbf{y} - \mathbf{X}\beta\|_2^2}_{\text{fit}} + \lambda \underbrace{\Omega(\beta)}_{\text{penalty}}$$

Ridge (ℓ_2): $\Omega = \|\beta\|_2^2$. Closed-form solution $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$. Shrinks all coefficients smoothly; never sets them exactly to zero. Best when many features each contribute a small amount (dense signal). Handles multicollinearity by stabilising $\mathbf{X}^\top \mathbf{X}$.

Lasso (ℓ_1): $\Omega = \|\beta\|_1$. No closed form; solved with coordinate descent or ISTA/ADMM. Produces sparse solutions (many coefficients exactly zero) due to the non-differentiability at the origin. Best

for automatic feature selection when you suspect only a few features matter.

Elastic Net: $\Omega = \alpha \|\beta\|_1 + (1 - \alpha) \|\beta\|_2^2$. Combines both penalties. Selects sparse groups of correlated features (Lasso alone arbitrarily picks one from a correlated set). Use when $p \gg n$ and features are correlated.

Hyperparameter selection: always use cross-validation over a log-spaced grid of λ . Standardise features first — ℓ_1/ℓ_2 penalties are not scale-invariant.

Ridge vs Lasso -- Python Example

```
import numpy as np
from sklearn.linear_model import RidgeCV, LassoCV, ElasticNetCV
from sklearn.datasets import make_regression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Synthetic dataset: 200 samples, 50 features, only 10 informative
X, y = make_regression(n_samples=200, n_features=50,
                      n_informative=10, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

alphas = np.logspace(-3, 3, 50)

ridge = RidgeCV(alphas=alphas, cv=5).fit(X_train, y_train)
lasso = LassoCV(alphas=alphas, cv=5, max_iter=10_000).fit(X_train, y_train)
enet = ElasticNetCV(l1_ratio=[.1, .5, .9, .99], alphas=alphas,
                   cv=5, max_iter=10_000).fit(X_train, y_train)

for name, model in [("Ridge", ridge), ("Lasso", lasso), ("ElasticNet", enet)]:
    r2 = model.score(X_test, y_test)
    nnz = np.sum(model.coef_ != 0)
    print(f"{name:12s} R2={r2:.4f} non-zero coefs={nnz}/50")
```

3 Classification

Chapter Overview

Classification assigns discrete labels to inputs. The choice of algorithm depends on data size, feature types, interpretability requirements, and the cost of different error types. Logistic regression provides probabilistic outputs and is highly interpretable; tree-based ensembles (Random Forest, XGBoost) are powerful on tabular data; neural networks excel on unstructured data. Evaluating classifiers requires going beyond accuracy: precision, recall, F1, AUC-ROC, and calibration curves each illuminate different failure modes.

Q6 — Explain logistic regression: the model, training objective, and its probabilistic interpretation.

Q: Derive the logistic regression training objective from maximum likelihood.

Answer:

Logistic regression models the posterior probability of class 1 given input $\mathbf{x}$:

$$P(y = 1 \mid \mathbf{x}; \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^\top \mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^\top \mathbf{x}}}$$

The log-likelihood over N i.i.d. samples is:

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^N [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

Maximising ℓ is equivalent to minimising the **binary cross-entropy loss**. Unlike OLS, there is no closed-form solution; optimisation uses gradient descent or Newton–Raphson (L-BFGS), both of which converge to the global optimum because the objective is convex.

✓ Best Practice

Calibration matters. The sigmoid output is only well-calibrated when the training distribution matches the deployment distribution and the model is not over- or under-regularised. Use Platt scaling or isotonic regression to calibrate any classifier before using its score as a probability in a downstream decision system.

Q7 — When would you choose Random Forest over XGBoost, or vice versa?

Q: Compare the two ensemble methods on training time, hyperparameters, and use-case fit.

Answer:

Both are tree ensembles that dominate tabular ML benchmarks, but they differ in mechanism:

Random Forest (bagging + feature randomness): Trains many deep trees independently in parallel, each on a bootstrap sample and a random feature subset. Predictions are averaged (regression) or majority-voted (classification). Robust to noisy features. Tolerant of default hyperparameters. Scales trivially across CPU cores. *Choose when:* you need a strong, interpretable baseline quickly; training data has many redundant or noisy features; hardware is limited.

XGBoost / LightGBM / CatBoost (gradient boosting): Trains shallow trees sequentially, each fitting the negative gradient of the loss from the previous ensemble. The sequential nature gives higher accuracy on most Kaggle-style datasets but makes it more sensitive to hyperparameters (learning rate, tree depth, subsampling ratios) and harder to parallelise. LightGBM's leaf-wise growth and histogram binning make it the fastest on large datasets. *Choose when:* raw predictive performance is paramount; you have the budget to tune hyperparameters; dataset is large and tabular.

From the Field

In production ML pipelines for structured data, my default is: prototype with Random Forest for a quick, robust baseline; switch to LightGBM with Optuna-tuned hyperparameters when squeezing out the last few percent of AUC matters for business metrics.

Q8 — Explain precision, recall, F1, and AUC-ROC. How do class imbalance and decision threshold affect them?

Q: Define each metric and state when you would prioritise one over another.

Answer:

Given confusion matrix entries TP, FP, TN, FN:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot P \cdot R}{P + R}$$

AUC-ROC plots TPR vs FPR at every classification threshold and measures the probability that the model ranks a random positive above a random negative. It is threshold-independent and insensitive to class imbalance — making it the preferred reporting metric for imbalanced problems.

Threshold decisions: lowering the classification threshold increases recall (catch more positives) at the cost of precision (more false alarms). The right threshold depends on the cost asymmetry: in cancer screening, a missed diagnosis (false negative) is far more costly than a false alarm, so you tune for high recall. In spam filtering, user experience degrades with false positives, so you tune for high precision.

! Pitfall / Warning

Accuracy is misleading under class imbalance. A classifier that always predicts the majority class achieves 99% accuracy on a dataset with 1% positives — and zero recall. Always report precision/recall/F1 or AUC for imbalanced problems. Consider also **AUC-PR** (precision–recall curve area), which is more informative than AUC-ROC when positives are rare.

4 Decision Trees and Ensemble Methods

Chapter Overview

Decision trees partition the feature space with axis-aligned splits. A single tree is interpretable but has high variance — small changes in data can produce very different trees. Ensemble methods — bagging, boosting, and stacking — reduce variance or bias by combining many trees. Understanding the splitting criteria (Gini impurity, entropy, MSE) and stopping conditions (depth, min-leaf-samples) is essential for both trees and forests.

Q9 — How does a decision tree choose which feature and split point to use?

Q: Explain the splitting criteria for classification (Gini/entropy) and regression (MSE), and discuss tree pruning.

Answer:

At each node, the tree evaluates all $(d \times B)$ candidate (feature, threshold) pairs and selects the one that maximises the **information gain** (reduction in impurity/error):

$$\Delta I = I(\text{parent}) - \frac{n_L}{n} I(L) - \frac{n_R}{n} I(R)$$

Impurity measures:

- **Gini impurity:** $I(t) = 1 - \sum_k p_k^2$. Ranges 0 (pure) to $1 - 1/K$ (uniform). Computationally cheaper than entropy; standard in sklearn.

- **Entropy / Information gain:** $I(t) = -\sum_k p_k \log_2 p_k$. Slightly favours balanced splits. Preferred in ID3/C4.5.
- **MSE (regression):** $I(t) = \frac{1}{n} \sum_i (y_i - \bar{y})^2$. Minimising variance within leaf regions.

Pruning strategies:

Pre-pruning (early stopping): Limit tree depth, require minimum samples per split/leaf, or only split if gain $> \epsilon$.

Post-pruning (cost-complexity): Grow full tree, then prune back by penalising complexity: $R_\alpha(T) = R(T) + \alpha|T|$ where $|T|$ is the number of leaves. Tune α with cross-validation.

Q10 — Explain gradient boosting from first principles. What problem does it solve that AdaBoost does not?

Q: Derive the gradient boosting update and compare to AdaBoost.

Answer:

Gradient Boosting (Friedman 2001) frames boosting as **functional gradient descent**: at step m , fit a tree h_m to the negative gradient of the loss $\mathcal{L}$ with respect to the current prediction $F_{m-1}(\mathbf{x})$:

$$r_i^{(m)} = - \left[\frac{\partial \mathcal{L}(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F=F_{m-1}}$$

Then update: $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu \cdot h_m(\mathbf{x})$, where ν is the learning rate (shrinkage).

Key insight: because the target for tree m is the negative gradient of *any* differentiable loss, gradient boosting handles regression, classification, ranking, and custom losses (quantile, Huber) with the same algorithm. AdaBoost is equivalent to gradient boosting with exponential loss, which makes it sensitive to outliers — gradient boosting with Huber or log loss is far more robust.

✓ Best Practice

Modern implementations (XGBoost, LightGBM) add second-order Taylor expansion of the loss (Newton boosting), column/row subsampling, and histogram binning. These additions give 10–100× speedups over naive gradient boosting with better accuracy. Always start with LightGBM for tabular data.

5 Support Vector Machines

Chapter Overview

Support Vector Machines (SVMs) find the maximum-margin hyperplane that separates classes. The **kernel trick** allows SVMs to operate implicitly in infinite-dimensional feature spaces without computing the feature map explicitly. While neural networks have displaced SVMs in most deep learning applications, SVMs remain competitive on small, high-dimensional datasets (text, genomics) and provide a clean mathematical framework for understanding margin-based learning.

Q11 — Derive the SVM primal and dual. What is the kernel trick?

Q: Walk through the hard-margin SVM optimisation, its Lagrangian dual, and explain kernels.

Answer:

Primal (hard margin): find $\mathbf{w}, b$ maximising the geometric margin $2/\|\mathbf{w}\|$, equivalently:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \quad \forall i$$

Introducing Lagrange multipliers $\alpha_i \geq 0$ and applying KKT stationarity conditions gives the **dual**:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \quad \text{s.t.} \quad \alpha_i \geq 0, \sum_i \alpha_i y_i = 0$$

The dual only depends on inputs through inner products $\mathbf{x}_i^\top \mathbf{x}_j$. **The kernel trick** replaces this with $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$ for some (possibly infinite-dimensional) feature map ϕ , without ever computing ϕ explicitly. Popular kernels: RBF $e^{-\gamma \|\mathbf{x} - \mathbf{x}'\|^2}$, polynomial $(1 + \mathbf{x}^\top \mathbf{x}')^d$, string kernels for text.

Soft-margin SVM introduces slack variables $\xi_i \geq 0$ and penalty $C \sum_i \xi_i$, trading margin width for training error tolerance. C is the primary regularisation hyperparameter.

6 Neural Networks and Deep Learning

Chapter Overview

Neural networks are universal function approximators composed of parameterised linear transformations followed by non-linear activations. Their power lies in **representation learning**: intermediate layers automatically construct hierarchical features from raw inputs. The modern deep learning stack — backpropagation, batch normalisation, residual connections, attention, and large-scale pretraining — has enabled breakthrough performance across vision, language, speech, and scientific domains.

Q12 — Explain backpropagation. Derive gradients for a two-layer network.

Q: Describe forward pass, loss computation, and gradient flow. Why does vanishing gradient occur?

Answer:

Backpropagation is the chain rule applied recursively to compute $\partial \mathcal{L} / \partial \theta_l$ for every parameter layer l . For a two-layer network with inputs $\mathbf{x}$, hidden layer $\mathbf{h} = \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1)$, and output $\hat{y} = \mathbf{W}_2 \mathbf{h} + b_2$:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} &= \delta_2 \mathbf{h}^\top, & \delta_2 &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} &= \delta_1 \mathbf{x}^\top, & \delta_1 &= \mathbf{W}_2^\top \delta_2 \odot \sigma'(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) \end{aligned}$$

Vanishing gradients: when σ' is small (e.g. sigmoid saturates at 0 or 1), δ_1 becomes exponentially small after many layers, and early layers stop learning. Solutions: ReLU activation (gradient = 1 for positive pre-activations), batch normalisation, residual connections (skip $\partial \mathcal{L} / \partial \mathbf{x}$ directly through identity path), and careful weight initialisation (He, Xavier).

Manual backprop for 2-layer network


```

import numpy as np

def relu(z):          return np.maximum(0, z)
def relu_grad(z):     return (z > 0).astype(float)

def forward(X, W1, b1, W2, b2):
    z1 = X @ W1.T + b1
    h  = relu(z1)
    z2 = h @ W2.T + b2
    return z1, h, z2

def backward(X, y, z1, h, z2, W1, W2):
    N = X.shape[0]
    # MSE loss gradient w.r.t. output
    dL_dz2 = 2 * (z2.squeeze() - y) / N    # (N,)
    dL_dW2 = dL_dz2[:, None].T @ h          # (out, hidden)
    dL_db2 = dL_dz2.sum()
    # Backprop through W2 and ReLU
    dL_dh  = dL_dz2[:, None] @ W2          # (N, hidden)
    dL_dz1 = dL_dh * relu_grad(z1)         # (N, hidden)
    dL_dW1 = dL_dz1.T @ X                  # (hidden, in)
    dL_db1 = dL_dz1.sum(axis=0)
    return dL_dW1, dL_db1, dL_dW2, dL_db2

```

Q13 — Compare Adam, SGD with momentum, and RMSProp. When should you prefer each?

Q: Explain the update rules and the practical trade-offs in convergence speed and generalisation.

Answer:

SGD + Momentum $\mathbf{v} \leftarrow \beta \mathbf{v} - \eta \nabla \mathcal{L}$; $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \mathbf{v}$. Accumulates gradient direction; escapes shallow local minima. Sensitive to learning rate choice. Often finds flatter minima than adaptive methods, which empirically generalise better on held-out data. *Default for large-scale vision tasks (ImageNet training).*

RMSProp $\mathbf{v} \leftarrow \beta \mathbf{v} + (1 - \beta)(\nabla \mathcal{L})^2$; $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{\sqrt{\mathbf{v} + \epsilon}} \nabla \mathcal{L}$. Adapts learning rate per-parameter by dividing by a running RMS of gradients. Excels on non-stationary objectives (RNNs).

Adam Combines momentum (first moment $\mathbf{m}$) with RMSProp (second moment $\mathbf{v}$), with bias correction. Effective across many architectures with minimal tuning. However, it can converge to sharper minima that generalise less well than SGD. *Default choice for Transformers and most NLP tasks.*

✓ Best Practice

Use Adam with a warmup + cosine annealing schedule for Transformers. Switch to SGD with momentum and a step-decay schedule for ResNets if you need maximum test accuracy. Tune learning rate first (most impactful hyperparameter), then weight decay.

7 Convolutional Neural Networks

Chapter Overview

Convolutional Neural Networks (CNNs) exploit the spatial structure of images through parameter sharing and local connectivity. A convolution operation slides a small filter across the input, computing dot products at each position and producing a feature map. Stacking convolutions, pooling, batch normalisation, and non-linearities builds up a hierarchy of features: edges and textures in early layers, object parts and semantics in later layers. CNNs revolutionised computer vision and remain foundational even as Vision Transformers (ViTs) increasingly challenge them at scale.

Q14 — How does a convolution operation work, and why is parameter sharing important?

Q: Describe the convolution computation, receptive field, and the benefit of weight sharing.

Answer:

A 2D convolution with kernel $\mathbf{K} \in \mathbb{R}^{k \times k}$ applied to input feature map $\mathbf{X} \in \mathbb{R}^{H \times W}$ produces output:

$$\mathbf{Y}[i, j] = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} \mathbf{X}[i+u, j+v] \cdot \mathbf{K}[u, v]$$

Output spatial dimensions: $H_{\text{out}} = \lfloor (H + 2p - k)/s \rfloor + 1$ (padding p , stride s).

Parameter sharing: the same k^2 kernel weights are used at every spatial location. This gives two benefits: (1) *drastically fewer parameters* vs. a fully connected layer (e.g. a 3×3 filter applied to a 256×256 image uses 9 weights instead of $256^2 \approx 65,000$); (2) *translation equivariance* — a shifted input produces a shifted output, which is exactly the right inductive bias for image recognition.

Receptive field of a unit in layer l is the input region that influences it. With kernel size k and stride 1, receptive field grows as $(l(k-1) + 1)^2$. Dilated convolutions and pooling layers grow it faster without adding parameters.

8 Recurrent Networks and Sequence Models

Chapter Overview

Recurrent Neural Networks (RNNs) process sequences by maintaining a hidden state $\mathbf{h}_t$ that summarises history up to step t . Vanilla RNNs suffer from vanishing gradients over long sequences. LSTMs and GRUs introduce gating mechanisms that allow gradients to flow over hundreds of steps. Since 2017, Transformer-based models have displaced RNNs in most sequence tasks by processing the entire sequence in parallel through self-attention. Understanding RNNs remains important for time-series tasks and edge deployment where compute is constrained.

Q15 — Why do vanilla RNNs fail on long sequences, and how do LSTMs fix this?

Q: Describe the vanishing gradient problem in RNNs and the LSTM gating mechanism.

Answer:

The RNN update is $\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t)$. Backpropagating through T steps multiplies Jacobians:

$$\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_1} = \prod_{t=2}^T \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \prod_{t=2}^T \text{diag}[\tanh'(\cdot)] \mathbf{W}_h$$

When $\|\mathbf{W}_h\| < 1$ or $\tanh' < 1$, this product shrinks exponentially — **vanishing gradients**. When $\|\mathbf{W}_h\| > 1$ it can explode (handled by gradient clipping).

LSTM (Hochreiter & Schmidhuber, 1997) introduces a **cell state** $\mathbf{c}_t$ that runs in parallel with additive (not multiplicative) updates, allowing gradients to flow:

$$\begin{aligned} \mathbf{f}_t &= \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f) && \text{(forget gate)} \\ \mathbf{i}_t &= \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i) && \text{(input gate)} \\ \tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_c) && \text{(candidate cell)} \\ \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t && \text{(cell update)} \\ \mathbf{o}_t &= \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o) && \text{(output gate)} \\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t) && \text{(hidden state)} \end{aligned}$$

The key insight: gradient flows through $\mathbf{c}_t \leftarrow \mathbf{c}_{t-1}$ via multiplication by $\mathbf{f}_t \in (0, 1)$, which can stay close to 1 for relevant memory, preventing collapse over hundreds of steps.

9 Transformers and Attention

Chapter Overview

The Transformer (Vaswani et al., 2017) replaced recurrence with **self-attention**: each token attends to all other tokens simultaneously, enabling parallelism during training and capturing long-range dependencies without gradient pathology. Combined with positional encodings, residual connections, and layer normalisation, Transformers scale predictably with data and compute — the foundation of GPT, BERT, T5, and every large language model in production today.

Q16 — Derive scaled dot-product attention. Why the scaling factor $\sqrt{d_k}$?

Q: Write out the attention formula and explain multi-head attention.

Answer:

Given queries $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$, keys $\mathbf{K} \in \mathbb{R}^{m \times d_k}$, values $\mathbf{V} \in \mathbb{R}^{m \times d_v}$:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

Why $\sqrt{d_k}$? The dot product $\mathbf{q}^\top \mathbf{k}$ has variance d_k when $\mathbf{q}, \mathbf{k}$ are unit-variance random vectors. Without scaling, the softmax receives inputs with large magnitude, producing near-one-hot attention distributions and near-zero gradients (softmax saturation). Dividing by $\sqrt{d_k}$ restores unit variance and ensures informative gradients throughout training.

Multi-head attention runs H parallel attention heads with separate learned projections $\mathbf{W}_i^Q, \mathbf{W}_i^K, \mathbf{W}_i^V$ then concatenates:

$$\text{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) \mathbf{W}^O$$

Each head can attend to different aspects of the input (syntax, coreference, semantics), and the concatenation enriches the representation at no extra depth.

✓ Best Practice

Flash Attention (Dao et al., 2022) reimplements attention with IO-aware tiling to avoid materialising the full $n \times n$ attention matrix in HBM, reducing memory from $O(n^2)$ to $O(n)$ and achieving 2–4× speedups. Always use Flash Attention for sequences longer than ~512 tokens in production.

10 Unsupervised Learning and Dimensionality Reduction

Chapter Overview

Unsupervised methods extract structure from unlabelled data. **Clustering** (k-means, DBSCAN, hierarchical) groups similar instances. **Dimensionality reduction** (PCA, t-SNE, UMAP, autoencoders) projects data to lower-dimensional manifolds for visualisation, compression, and feature extraction. **Density estimation** (GMMs, normalising flows, VAEs) models the data distribution for generation, anomaly detection, and imputation. These techniques are increasingly important as labelled data remains scarce and self-supervised pretraining on unlabelled data dominates frontier ML.

Q17 — Explain the k-means algorithm. What are its limitations and how does k-means++ address initialisation?

Q: Describe the Lloyd’s algorithm update steps, convergence guarantees, and practical failure modes.

Answer:

Lloyd’s algorithm (k-means):

1. Initialise K centroids $\mu_1, \dots, \mu_K$.
2. **Assignment step:** assign each point to its nearest centroid: $c_i = \arg \min_k \|\mathbf{x}_i - \mu_k\|^2$.
3. **Update step:** recompute centroids as cluster means: $\mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$.
4. Repeat until assignments do not change.

The objective $J = \sum_i \|\mathbf{x}_i - \mu_{c_i}\|^2$ (within-cluster sum of squares) decreases monotonically and is bounded below, so convergence is guaranteed — to a *local* minimum.

Limitations: assumes spherical clusters of equal size; sensitive to outliers (squared distance penalises them heavily); requires pre-specifying K ; deterministic results depend on initialisation.

k-means++ initialisation: choose first centroid uniformly at random; each subsequent centroid is chosen with probability proportional to its squared distance from the nearest existing centroid. This spreads centroids across the data and provides an $O(\log K)$ approximation guarantee to the optimal objective.

11 Feature Engineering and Preprocessing

Chapter Overview

In industrial ML systems, feature engineering — transforming raw data into informative representations — often determines model quality more than algorithm choice. This chapter covers scaling, encoding, handling missing data, and creating informative features. A well-engineered feature set on a simple model regularly outperforms a poorly prepared feature set on a complex one.

Q18 — How do you handle missing data? Compare deletion, imputation, and model-based approaches.

Q: Classify missing data mechanisms and describe appropriate handling strategies.

Answer:

Missing data mechanisms (Rubin, 1976):

MCAR (Missing Completely At Random): Missingness is independent of observed and missing values. Complete-case analysis (listwise deletion) is unbiased but reduces sample size.

MAR (Missing At Random): Missingness depends only on observed values. Imputation conditional on observed values is valid.

MNAR (Missing Not At Random): Missingness depends on the missing value itself (e.g. high earners omit salary). Requires domain-specific handling; imputation alone introduces bias.

Strategies:

- **Deletion:** safe only under MCAR and when missingness rate is low ($< 5\%$).
- **Simple imputation:** mean/median (continuous), mode or “Unknown” category (categorical). Fast; can distort distributions and underestimate variance.
- **Multiple imputation (MICE):** create M complete datasets with different imputed values, train model on each, pool results. Correctly propagates uncertainty; computationally expensive.
- **Model-based:** train a regressor/classifier to predict missing values from observed ones. KNN imputation is effective when data has local structure.
- **Indicator variable:** add a binary “was_missing” feature alongside imputed value. Lets the model learn from the missingness pattern itself — especially useful under MNAR.

! Pitfall / Warning

Always fit your imputer on the training set and *transform* the validation/test sets using training statistics. Fitting on the full dataset causes data leakage and will give optimistic evaluation metrics.

12 Model Evaluation and Selection

Chapter Overview

Rigorous evaluation is what separates a research experiment from a production system. This chapter covers cross-validation strategies, the correct use of train/val/test splits, hyperparameter optimisation, and the statistical tests needed to determine whether performance differences are real or random.

Q19 — Explain k-fold cross-validation. When would you use stratified, grouped, or time-series CV?

Q: Describe the CV procedure and state when each variant is appropriate.

Answer:

Standard k-fold CV: randomly partition data into k equal folds. Train on $k - 1$ folds, evaluate on the held-out fold, rotate, and average the k scores. Typical $k = 5$ or 10 balances bias (larger $k \rightarrow$ lower bias in estimate) and variance (larger $k \rightarrow$ higher variance, more compute).

When to use variants:

Stratified k-fold: Maintains class proportions in each fold. Essential for imbalanced classification (e.g. 1% fraud rate); random splits could put zero positives in a fold.

Group k-fold: Ensures all records from a given entity (patient, user, store) appear in only one fold. Use when records within a group are correlated (repeated measurements). Prevents data leakage from the same person appearing in train and test.

Time-series / walk-forward CV: Train on $t_1, \dots, t_k$, validate on $t_{k+1}, \dots, t_{k+h}$, advance window. Respects temporal ordering; prevents future data leaking into training. Use for any time-series forecasting or temporal prediction task.

13 Bayesian Methods

Chapter Overview

Bayesian methods frame learning as probabilistic inference: we maintain a prior over model parameters, update it with observed data to obtain a posterior, and integrate over the posterior to make predictions. This principled uncertainty quantification is invaluable in high-stakes applications (medicine, autonomous driving, finance) where knowing *what the model does not know* is as important as its point predictions.

Q20 — Explain Bayesian inference. What is the trade-off between Bayesian and frequentist approaches?

Q: State Bayes' theorem in the context of ML and discuss practical inference challenges.

Answer:

Given data $\mathcal{D}$ and parameters θ , Bayes' theorem gives:

$$\underbrace{p(\theta | \mathcal{D})}_{\text{posterior}} = \frac{\underbrace{p(\mathcal{D} | \theta)}_{\text{likelihood}} \cdot \underbrace{p(\theta)}_{\text{prior}}}{\underbrace{p(\mathcal{D})}_{\text{marginal likelihood}}}$$

Prediction integrates over the posterior: $p(y^* | \mathbf{x}^*, \mathcal{D}) = \int p(y^* | \mathbf{x}^*, \theta) p(\theta | \mathcal{D}) d\theta$.

Advantages of Bayesian approach: principled uncertainty quantification; automatically regularises via the prior; small-data regime performance; can incorporate domain knowledge in the prior; naturally handles model comparison via marginal likelihood.

Challenges: the marginal likelihood $p(\mathcal{D}) = \int p(\mathcal{D} | \theta) p(\theta) d\theta$ is intractable for most models. Practical solutions: Laplace approximation, variational inference (ELBO), Markov Chain Monte Carlo (MCMC), or approximate methods like Bayesian neural networks with MC Dropout.

Frequentist methods (MLE, OLS, hypothesis testing) are simpler, computationally cheap, and work well with large data. Bayesian methods shine when data is scarce, uncertainty is critical, or sequential

updating is needed (online learning, A/B tests with early stopping).

14 Natural Language Processing

Chapter Overview

NLP has undergone two revolutions: the first was word embeddings (Word2Vec, GloVe) that gave words continuous vector representations; the second was the Transformer, which enabled models trained on internet-scale corpora to be fine-tuned with a handful of labelled examples for almost any task. Modern NLP is dominated by large language models (LLMs): BERT for understanding, GPT for generation, T5 for seq2seq, and their successors.

Q21 — Explain word embeddings: Word2Vec, GloVe, and contextual embeddings (BERT). What are their key differences?

Q: Describe how each representation is trained and when you would use each.

Answer:

Word2Vec (Mikolov et al., 2013): Static embeddings trained with a shallow neural network on local context windows. Two objectives: CBOW (predict centre word from context) and Skip-gram (predict context from centre word). Captures syntactic and semantic regularities: $\text{king} - \text{man} + \text{woman} \approx \text{queen}$. *Limitation:* a word has one vector regardless of context ("bank" as riverbank vs. financial institution).

GloVe (Pennington et al., 2014): Factorises the global word co-occurrence matrix. Combines global statistics (like LSA) with the local context structure of Word2Vec. Often slightly better on analogy and similarity benchmarks.

BERT contextual embeddings: A Transformer encoder pre-trained with Masked Language Modelling (MLM) and Next Sentence Prediction (NSP) on large corpora. Each token receives a *context-dependent* representation: "bank" in two different sentences produces two different vectors. Fine-tuning on downstream tasks achieves state-of-the-art across NLU benchmarks.

When to use: Word2Vec/GloVe are lightweight feature extractors for simple text classifiers (TF-IDF + LogReg) or when compute is constrained. BERT-family models are the default for any task requiring deep language understanding.

15 Reinforcement Learning

Chapter Overview

Reinforcement learning (RL) trains agents to maximise cumulative reward through trial and error in an environment. The Markov Decision Process (MDP) framework formalises the setting: states, actions, transition dynamics, and a reward function. Key algorithms divide into value-based methods (Q-learning, DQN), policy gradient methods (REINFORCE, PPO, SAC), and model-based RL. RL powers game-playing agents (AlphaGo, Dota2), robotics, recommender systems, and RLHF for aligning LLMs.

Q22 — Explain the Bellman equation and Q-learning. Why is experience replay and a target network needed in DQN?

Q: Derive the Bellman optimality equation and describe the DQN stabilisation tricks.

Answer:

The **Bellman optimality equation** defines the optimal action-value function $Q^*(s, a)$:

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]$$

Q-learning tabularly iterates: $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$. This is off-policy (behaviour policy can differ from greedy policy) and model-free (no transition dynamics needed).

DQN (Mnih et al., 2015) approximates $Q(s, a; \theta)$ with a neural network. Two instability sources arise and two tricks fix them:

Experience replay: Store (s, a, r, s') transitions in a replay buffer; sample mini-batches i.i.d. during training. Breaks temporal correlations that would make gradient estimates highly correlated and biased.

Target network: Maintain a frozen copy θ^- of the network updated every C steps. Compute TD targets as $r + \gamma \max_{a'} Q(s', a'; \theta^-)$ instead of using θ itself. Prevents the bootstrap target from “chasing its own tail” and causing divergence.

16 MLOps and Production ML

Chapter Overview

MLOps bridges model development and reliable production deployment. A model that achieves 95% accuracy in a notebook is worthless if it cannot be served reliably, monitored for drift, retrained automatically, and explained to stakeholders. This chapter covers the deployment lifecycle: model packaging (Docker, ONNX), serving (REST APIs, batch inference), monitoring (data drift, concept drift, performance degradation), and CI/CD pipelines for ML.

Q23 — What is data drift and concept drift? How do you detect and respond to each?

Q: Define both drift types, describe detection methods, and outline a monitoring architecture.

Answer:

Data drift (covariate shift): The distribution of input features $P(\mathbf{x})$ changes between training and serving, while the conditional $P(y|\mathbf{x})$ remains stable. Example: a fraud model trained before a pandemic sees very different transaction patterns post-pandemic.

Concept drift: The relationship $P(y|\mathbf{x})$ itself changes. Example: customer preferences shift with market trends; yesterday’s high-quality lead becomes today’s disinterested prospect.

Detection methods:

- **Statistical tests on input features:** Population Stability Index (PSI), Kolmogorov–Smirnov test, Maximum Mean Discrepancy (MMD) between training distribution and recent serving data.

- **Performance monitoring:** track accuracy, precision, recall, AUC on labelled production data (where ground truth is available with delay). Alert when metrics drop below a threshold.
- **Proxy metrics:** if labels arrive with delay, monitor prediction distribution shift as an early warning.

Response:

- **Retraining schedule:** periodic (weekly/monthly) or triggered by drift alerts.
- **Incremental learning:** update model weights on a sliding window of recent data.
- **Ensemble with recency bias:** blend old and freshly-trained models, upweighting the newer one as confidence grows.

✓ Best Practice

Build a **drift monitoring pipeline** as a first-class citizen alongside your serving infrastructure. Log every prediction with its input features and prediction timestamp. Use a tool like Evidently AI, WhyLogs, or a custom pipeline on your data warehouse to compute drift metrics daily and push alerts to Slack/PagerDuty.

Q24 — Describe a production ML serving architecture. How do you handle latency, throughput, and reliability?

Q: Design a low-latency ML API service and discuss the engineering trade-offs.

Answer:

A robust serving architecture separates concerns across four layers:

1. **Model registry:** versioned model artefacts stored in S3/GCS with metadata (training date, data version, performance metrics). Tools: MLflow, Weights & Biases Registry, SageMaker Model Registry.
2. **Serving layer:** containerised inference server (TorchServe, Triton Inference Server, or a custom FastAPI wrapper) behind a load balancer. Export models to ONNX or TensorRT for hardware-optimised inference. Batch inference requests (dynamic batching) to amortise fixed GPU overhead.
3. **Feature store:** pre-computed, low-latency feature serving (Redis, Feast) for features that are expensive to compute at request time. Ensures training-serving consistency — the same feature computation code is used in both pipelines.
4. **Monitoring and logging:** capture input features, prediction scores, and ground-truth labels (when available) to an event stream (Kafka) for drift detection, debugging, and offline analysis.

Latency trade-offs:

- Use **model quantisation** (INT8/FP16) and **pruning** to reduce model size and inference time.
- Cache predictions for high-frequency, low-cardinality inputs (e.g. item popularity scores).
- Set **P99 latency SLOs** (e.g. <50ms), not just average latency. Tail latency determines user experience.

17 Explainability and Ethics in ML

Chapter Overview

As ML systems make consequential decisions — credit, hiring, medical diagnosis, criminal justice — the ability to explain *why* a model made a prediction is both a regulatory requirement (GDPR Article 22, EU AI Act) and a scientific necessity. This chapter covers model-agnostic explanation methods (SHAP, LIME), intrinsically interpretable models, and the ethical considerations around fairness, accountability, and transparency.

Q25 — Explain SHAP values. Why are they theoretically grounded compared to other feature importance methods?

Q: Define SHAP, derive the Shapley value axioms, and compare to permutation importance and LIME.

Answer:

SHAP (SHapley Additive exPlanations, Lundberg & Lee 2017) assigns each feature j a value ϕ_j representing its contribution to the prediction $f(\mathbf{x})$ relative to a baseline $\mathbb{E}[f(\mathbf{X})]$:

$$f(\mathbf{x}) = \mathbb{E}[f(\mathbf{X})] + \sum_{j=1}^p \phi_j$$

SHAP values are the unique attribution satisfying four axioms from cooperative game theory:

- **Efficiency:** $\sum_j \phi_j = f(\mathbf{x}) - \mathbb{E}[f(\mathbf{X})]$ (attributions sum to prediction).
- **Symmetry:** equally contributing features get equal attributions.
- **Dummy:** a feature that never changes the prediction gets attribution 0.
- **Linearity:** attribution over a mixture of models equals the mixture of attributions.

The Shapley value for feature j averages the marginal contribution of j over all possible feature subsets S :

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(p - |S| - 1)!}{p!} [f_{\mathbf{x}}(S \cup \{j\}) - f_{\mathbf{x}}(S)]$$

Comparison:

Permutation importance: Measures drop in accuracy when feature is shuffled. Fast; global; can underestimate importance of correlated features.

LIME: Fits a local linear model around the instance. Simple to explain; unstable (results vary with sampling); not additive.

SHAP: Theoretically grounded; consistent; local and global; computationally expensive for exact computation but TreeSHAP is $O(TLD^2)$ for tree ensembles.

Q&A Drill: 200+ Interview Questions

Topic-Classified Rapid-Fire Q&A Collection

★ Key Insight

This section contains 200+ interview questions across 14 topic-classified chapters. Each answer is concise and interview-ready. Questions with Python examples include runnable code. Use this section for rapid self-testing and revision.

18 Q&A — Foundations & Learning Paradigms

QQ1 — What is supervised learning?

Answer. Supervised learning trains on labelled data where each example has input–output pairs. The model learns to map inputs to outputs. Examples: classification (spam detection), regression (house price prediction).

Python

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)
print(clf.score(X_test, y_test))
```

QQ2 — What is unsupervised learning?

Answer. Unsupervised learning finds hidden patterns in unlabelled data, commonly used for clustering or dimensionality reduction.

Python

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
iris = load_iris()
X = iris.data
kmeans = KMeans(n_clusters=3)
kmeans.fit(X)
print(kmeans.labels_)
```

QQ3 — What is overfitting, and how can you prevent it?

Answer. Overfitting happens when a model learns noise and details from training data to the extent that it degrades performance on new data. Prevention: regularisation, more data, feature selection, cross-validation, early stopping, or simplifying the model.

QQ11 — What is regularisation in machine learning?

Answer. Regularisation reduces model complexity and prevents overfitting by adding a penalty on model parameters. Common types: L1 (Lasso) and L2 (Ridge).

Python

```
from sklearn.linear_model import Ridge
ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
print(ridge.score(X_test, y_test))
```

QQ12 — What is the bias–variance tradeoff?

Answer. The bias–variance tradeoff balances a model that is too simple (high bias, underfits) against one that is too complex (high variance, overfits). The goal is to find an optimal model with both low bias and low variance.

QQ15 — What is feature selection?

Answer. Feature selection is choosing the most important variables for model training to prevent overfitting, reduce computation, and improve interpretability.

Python

```
from sklearn.feature_selection import SelectKBest, f_classif
selector = SelectKBest(score_func=f_classif, k=2)
X_new = selector.fit_transform(X, y)
```

QQ17 — How does Random Forest work?

Answer. Random Forest is an ensemble method that constructs multiple decision trees on bootstrapped samples and averages their predictions to improve accuracy and reduce overfitting.

Python

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100)
rf.fit(X_train, y_train)
print(rf.score(X_test, y_test))
```

QQ18 — What is Support Vector Machine (SVM)?

Answer. SVM is a supervised algorithm for classification and regression that finds the optimal hyperplane maximising the margin between different classes.

Python

```
from sklearn.svm import SVC
svm = SVC(kernel='linear')
svm.fit(X_train, y_train)
```

QQ21 — What are neural networks?

Answer. Neural networks are computational models inspired by the human brain, consisting of interconnected layers of nodes (neurons). Used for both supervised and unsupervised learning, especially powerful for complex tasks.

QQ23 — What is dropout in neural networks?

Answer. Dropout is a regularisation technique where a fraction of neurons are randomly set to zero during training, preventing overfitting.

Python

```
from keras.layers import Dropout
model.add(Dropout(0.5))
```

QQ29 — What is early stopping?

Answer. Early stopping ends model training when performance on a validation set stops improving, preventing overfitting.

QQ40 — What is pruning in decision trees?

Answer. Pruning removes parts of a tree that do not provide additional predictive power, avoiding overfitting.

QQ52 — Give an example of transfer learning.

Answer. Using a pre-trained VGG16 model (from ImageNet) for feature extraction in a custom image classification problem.

Python

```
from tensorflow.keras.applications import VGG16
model = VGG16(weights='imagenet', include_top=False)
```

QQ53 — What is an autoencoder?

Answer. An autoencoder is an unsupervised neural network for learning efficient (compressed) data representations, typically for dimensionality reduction or denoising.

QQ74 — What is early stopping in neural networks? (revisited)

Answer. A regularisation strategy that halts training when validation performance stops improving, preventing overfitting.

QQ85 — What is dropout in neural networks? (revisited)

Answer. A regularisation method where random neurons are zeroed during training to reduce overfitting and improve generalisation.

QQ110 — Difference between supervised and unsupervised learning?

Answer. Supervised learning uses labelled data; unsupervised learning finds structure in unlabelled data.

QQ111 — What is feature drift?

Answer. Feature drift occurs when the statistical properties of input features change over time, potentially degrading model performance.

QQ112 — What is concept drift?

Answer. Concept drift happens when the statistical relationship between input and output changes over time, impacting model predictions.

QQ113 — What is semi-supervised learning?

Answer. A learning paradigm where the model trains on a small amount of labelled data plus a large amount of unlabelled data.

QQ114 — What is active learning?

Answer. A technique where the model interactively queries the user or oracle to label new data points, improving learning efficiency.

QQ118 — What is label smoothing?

Answer. A regularisation technique where target labels are adjusted (smoothed) to avoid overconfidence in models, common in deep learning.

QQ135 — What is XGBoost and why is it popular?

Answer. XGBoost is an efficient, optimised gradient boosting implementation supporting parallelisation, regularisation, and missing value handling. It is popular because it delivers consistently high accuracy, flexibility, and handles large datasets.

Python

```
import xgboost as xgb
model = xgb.XGBClassifier()
model.fit(X_train, y_train)
```

QQ136 — What is regularisation (L1, L2) and how do they help?

Answer. L1 (Lasso): Adds absolute weight values to loss, encouraging sparsity and feature selection. **L2 (Ridge):** Adds squared weights to loss, discouraging large weights and reducing overfitting. Both prevent overfitting and improve generalisation.

Python

```
from sklearn.linear_model import Lasso, Ridge
lasso = Lasso(alpha=0.1)
ridge = Ridge(alpha=1.0)
```

QQ137 — How does early stopping improve generalisation?

Answer. Early stopping monitors validation performance during training and halts when it fails to improve for a set number of epochs (patience). This prevents the model from memorising training data. Tip: use with deep learning, boosting, and any iterative training with clear validation metrics.

QQ144 — What is transfer learning? (Advanced)

Answer. Transfer learning uses pre-trained models as starting points for new but related tasks. **Fine-tuning:** Unfreeze layers and retrain on task-specific data. **Feature extraction:** Use the pre-trained model as a fixed extractor. Achieves strong results with less data and compute.

QQ147 — What is label propagation in semi-supervised learning?

Answer. Label propagation assigns class labels to unlabelled points based on similarity to labelled examples—using a graph-based approach to spread known labels through the dataset.

QQ148 — What are variational autoencoders (VAEs)?

Answer. VAEs are generative unsupervised neural networks that encode input to a probabilistic latent space and decode to reconstruct data—enabling new sample generation, interpolation, and probabilistic reasoning.

QQ151 — What is transfer learning domain adaptation?

Answer. Domain adaptation specifically refers to transferring knowledge between differing but related data distributions—adjusting representations to maintain performance as distributions shift between source (training) and target (deployment) data.

QQ157 — Bagging vs. boosting in real-world use?

Answer. Bagging (e.g. Random Forest): Reduces variance and helps with overfitting by averaging many independent models. **Boosting** (e.g. XGBoost): Reduces bias by building sequential models, each correcting its predecessor's errors—often delivering stronger results for structured/tabular data.

QQ163 — What is self-supervised learning?

Answer. Self-supervised learning leverages data's own structure for supervision—models generate their own labels from input data (e.g. predicting masked words in BERT, or the next video frame). Applications: pre-training for NLP, vision, and audio.

QQ185 — What is active sampling in ML?

Answer. Active sampling is a data selection strategy where the model identifies data points whose labels would most improve it if known. It is critical when manual labelling is costly (medical images, legal documents). Techniques: uncertainty sampling, query-by-committee, expected model change, diversity sampling.

QQ186 — What is knowledge transfer in multi-task learning?

Answer. In multi-task learning (MTL), knowledge transfer occurs when learning one task improves performance in another by sharing representations or features. Benefit: better generalisation, reduced overfitting risk, data efficiency.

QQ188 — What is elastic net regularisation?

Answer. Elastic Net combines L1 (Lasso) and L2 (Ridge) penalties. L1 drives some coefficients to zero (feature selection); L2 shrinks all coefficients. Useful when there are many correlated predictors, balancing variable selection and model stability.

QQ190 — What is polynomial regression?

Answer. A regression type where input variables are raised to integer powers to capture non-linear trends. Risk: higher degrees capture more complexity but can overfit—regularisation or cross-validation is important.

QQ191 — What are the limitations of deep learning?

Answer.

- **Data intensive:** Requires large labelled datasets.
- **Computationally expensive:** High training and inference costs.
- **Lack of interpretability:** Often yields “black box” predictions.

- **Adversarial sensitivity:** Vulnerable to small crafted input changes.
- **Overfitting:** Can memorise without proper regularisation.
- **Deployment difficulties:** Hard to update or control large models.

QQ192 — What is active learning and when is it useful?

Answer. Active learning lets a model interactively query a human annotator for labels on the most uncertain or informative samples. Useful when labelling is expensive or scarce (e.g. medical AI, rare events) to maximise learning efficiency.

QQ194 — What is batch normalisation and why is it critical?

Answer. Batch normalisation normalises intermediate activations in a neural network, stabilising and accelerating training. Benefits: faster convergence, higher learning rates, regularisation effect, less sensitivity to initialisation.

QQ196 — What is dropout and how does it prevent overfitting?

Answer. Dropout randomly disables a proportion of neurons during each training pass, forcing redundancy in representation and discouraging any single pathway's dominance. Result: significant boost in test-time generalisation, especially in large networks.

QQ197 — What is early stopping? (concise)

Answer. Early stopping halts training when a monitored metric (e.g. validation loss) stops improving for a set number of iterations ("patience"). Reduces overfitting risk; preserves weights from the best-generalising checkpoint.

QQ201 — Why is transfer learning transformative for small datasets?

Answer. Transfer learning uses pre-trained models as starting points on new but related tasks. It achieves high accuracy with less data and computation, democratising advanced ML for small or domain-specific datasets.

QQ203 — What is feature drift and concept drift?

Answer. Feature drift: Changes in the distribution of predictor variables. **Concept drift:** Changes in the mapping from input features to the target variable. Impact: reduces model performance over time—continuous monitoring and retraining are essential in production.

19 Q&A — Data Preparation & Feature Engineering

QQ6 — What is feature scaling, and why is it important?

Answer. Feature scaling standardises the range of independent variables—especially important for algorithms like kNN and SVM. Common methods: Standardisation (z-score) and Min-Max scaling.

Python

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

QQ13 — Explain one-hot encoding.

Answer. One-hot encoding converts categorical variables into binary vectors—each category becomes a new column with values 0 or 1.

Python

```
import pandas as pd
df = pd.DataFrame({'color': ['red', 'blue', 'green']})
one_hot = pd.get_dummies(df['color'])
print(one_hot)
```

QQ25 — What is data normalisation?

Answer. Normalisation transforms features to have zero mean and unit variance, or to fit within a fixed range (e.g. 0 to 1).

Python

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
X_norm = scaler.fit_transform(X)
```

QQ26 — How do you handle missing values in data?

Answer. Common methods: removing rows, imputing with mean/median/mode, or forward/backward fill.

Python

```
import pandas as pd
df.fillna(df.mean(), inplace=True)
```

QQ41 — What is feature engineering?

Answer. Feature engineering creates new features or modifies existing ones from raw data to improve model performance.

QQ42 — Give examples of feature engineering.

Answer. Scaling, log transforms, encoding categoricals, extracting date/time features, interactions (products/sums), polynomial features.

QQ43 — What is label encoding?

Answer. Label encoding converts categorical labels into numeric codes for model use.

Python

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y_encoded = le.fit_transform(y)
```

QQ59 — What is DBSCAN?

Answer. DBSCAN is a density-based clustering algorithm that groups closely packed points and marks as outliers points lying alone in low-density regions.

QQ76 — What is batch normalisation?

Answer. A method normalising layer inputs for each mini-batch during training, improving speed and stability.

Python

```
from keras.layers import BatchNormalization
model.add(BatchNormalization())
```

QQ84 — What is data augmentation?

Answer. Creating new training samples by modifying existing ones (e.g. rotating or flipping images) to improve model robustness and generalisation.

QQ106 — What is an outlier?

Answer. A data point significantly different from others—can distort statistical analyses and model performance.

QQ121 — What is a data pipeline?

Answer. A series of data processing steps (cleaning, feature engineering, modelling, deployment) organised in sequence for reproducibility and automation.

QQ123 — What is data versioning and why is it important?

Answer. Tracking different versions of datasets used in ML experiments, enabling reproducibility and auditing of results.

QQ129 — What is feature hashing?

Answer. A feature engineering technique using a hash function to convert categorical variables into fixed-length vectors, often used in NLP.

QQ132 — What is cross-validation leakage and how do you avoid it?

Answer. Data leakage occurs when information from outside the training fold inflates evaluation scores (e.g. scaling before splitting). Avoid it by fitting transformations *inside* the cross-validation loop or using scikit-learn's Pipeline. Never peek at the test set during preprocessing.

QQ140 — How do you handle categorical variables in tree-based models?

Answer. Linear models: Require numeric input—use label or one-hot encoding. **Tree-based models:** Can handle label-encoded categorical variables natively. **Advanced:** CatBoost and LightGBM have specialised handling without explicit encoding.

QQ142 — What are positional encodings in Transformer models?

Answer. Transformers have no inherent notion of sequence, so positional encodings inject position/order information—typically sinusoidal or learned vectors added to input embeddings.

QQ153 — What is CatBoost and how does it handle categorical features?

Answer. CatBoost is a gradient boosting library designed to natively handle categorical variables without explicit encoding, using permutation-driven statistics during training.

QQ160 — What are Bayesian neural networks, and their advantage?

Answer. Bayesian neural networks add uncertainty estimation to weights and predictions, enabling better-calibrated confidence intervals, outlier detection, and robust decision making—especially in risk-sensitive applications like medicine and finance.

QQ167 — What is the Gumbel-softmax trick?

Answer. Enables differentiable sampling from a categorical distribution—crucial for training neural networks where discrete choices are involved, such as neural architecture search or NLP models.

QQ176 — What is robust regression?

Answer. Regression techniques less sensitive to outliers than ordinary least squares. Examples: RANSAC, Huber regression, Theil–Sen estimator.

QQ193 — What is the role of feature importance in ML?

Answer. Feature importance scores identify which features most influence predictions. Why important: guides data collection, feature engineering, trust in model decisions, and is used for interpretation, debugging, compliance, and scientific discovery.

20 Q&A — Model Evaluation, Validation & Experimentation

QQ7 — How do you evaluate a classification model?

Answer. Common metrics: Accuracy, Precision, Recall, F1-score, ROC-AUC. A confusion matrix gives detailed evaluation.

Python

```
from sklearn.metrics import classification_report, confusion_matrix
y_pred = clf.predict(X_test)
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

QQ8 — Explain cross-validation.

Answer. Cross-validation splits data into multiple folds—each used as a validation set while others train the model. Most common: k-fold cross-validation.

Python

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(clf, X, y, cv=5)
print(scores)
```

QQ9 — What is hyperparameter tuning?

Answer. Finding the best combination of model settings (e.g. tree depth, number of neighbours) to maximise performance.

Python

```
from sklearn.model_selection import GridSearchCV
params = {'max_depth': [3, 5, 7]}
grid = GridSearchCV(DecisionTreeClassifier(), params, cv=3)
grid.fit(X, y)
print(grid.best_params_)
```

QQ10 — What is a confusion matrix?

Answer. A confusion matrix summarises classification model performance showing true positive, false positive, true negative, and false negative counts.

Python

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

QQ19 — What is the purpose of the ROC curve?

Answer. The ROC curve plots classifier performance as the discrimination threshold varies—showing the trade-off between true positive rate and false positive rate.

Python

```
from sklearn.metrics import roc_curve, auc
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)
print(auc(fpr, tpr))
```

QQ20 — What is precision and recall?

Answer. Precision: Ratio of true positives to all predicted positives. **Recall:** Ratio of true positives to all actual positives.

QQ30 — What is model deployment?

Answer. Integrating a trained model into a production environment where it can make real-time predictions on new data.

QQ31 — What is ensemble learning?

Answer. Ensemble learning combines multiple models (often weak learners) to improve overall performance, accuracy, and robustness.

QQ36 — Explain gradient boosting.

Answer. Gradient boosting builds models sequentially—each new one trained on the residuals of the previous model, improving prediction accuracy.

Python

```
from sklearn.ensemble import GradientBoostingClassifier
gb = GradientBoostingClassifier()
gb.fit(X_train, y_train)
```

QQ47 — What are confusion matrix terms: TP, FP, TN, FN?

Answer. TP: True Positive — **FP:** False Positive — **TN:** True Negative — **FN:** False Negative.

QQ48 — What is the F1 score?

Answer. F1 score is the harmonic mean of precision and recall, balancing both in a single metric:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

QQ54 — What are convolutional neural networks (CNNs)?

Answer. CNNs are specialised neural networks for processing grid-like data (e.g. images), using convolutional layers to automatically extract hierarchical features.

QQ56 — What is a confusion matrix used for?

Answer. Visually assessing classification model performance by showing correct and incorrect predictions across all classes.

QQ60 — What is silhouette score?

Answer. Evaluates how similar a data point is to its own cluster versus other clusters, ranging from -1 to 1 (higher is better).

QQ62 — What is a ROC-AUC score?

Answer. Area Under the Receiver Operating Characteristic Curve—measures a classifier's ability to discriminate between classes (higher is better; 0.5 = random, 1.0 = perfect).

QQ66 — What is the precision-recall tradeoff?

Answer. The balance between precision (minimising false positives) and recall (minimising false negatives)—a model typically cannot maximise both simultaneously.

QQ90 — What is a Pipeline in scikit-learn?

Answer. A pipeline chains preprocessing steps and estimator(s) so the entire workflow can be treated as one composite model.

Python

```
from sklearn.pipeline import Pipeline
pipe = Pipeline([('scaler', StandardScaler()), ('clf', LogisticRegression())])
pipe.fit(X_train, y_train)
```

QQ92 — What is the Transformer architecture?

Answer. A deep learning model using self-attention to process sequences in parallel, highly effective for language tasks (e.g. GPT, BERT).

QQ98 — What is reinforcement learning?

Answer. A learning process where agents take actions in an environment to maximise a reward signal over time.

QQ100 — What is a Markov Decision Process (MDP)?

Answer. A framework for modelling decision-making, involving states, actions, rewards, and transition probabilities—the mathematical foundation for reinforcement learning.

QQ103 — What is data leakage?

Answer. When information from outside the training dataset is used to create the model, leading to overly optimistic evaluations that do not hold in production.

QQ109 — What is hyperparameter optimisation?

Answer. Systematic approach to finding the best hyperparameters for a model, using Grid Search, Random Search, or Bayesian Optimisation.

QQ116 — What is a precision-recall curve?

Answer. A plot visualising the trade-off between precision and recall for different probability thresholds—particularly useful for imbalanced datasets.

QQ117 — What is model calibration?

Answer. Adjusting a model's predicted probabilities to better reflect true likelihood, often with Platt scaling or isotonic regression.

QQ124 — What is a hyperparameter search space?

Answer. The range and values of all tunable parameters considered during hyperparameter optimisation.

QQ133 — What is stratified k-fold cross-validation?

Answer. Stratified k-fold keeps class distribution consistent across all folds, ensuring each split is representative of the whole dataset. Prefer for imbalanced classification tasks.

Python

```
from sklearn.model_selection import StratifiedKFold
skf = StratifiedKFold(n_splits=5)
for train_idx, val_idx in skf.split(X, y):
    X_train, X_val = X[train_idx], X[val_idx]
    y_train, y_val = y[train_idx], y[val_idx]
```

QQ134 — Explain the ROC curve, AUC, and their practical significance.

Answer. ROC curve: Plots True Positive Rate vs. False Positive Rate at various thresholds. **AUC:** Summarises the ROC curve (maximum = 1; higher = better discrimination). Particularly useful for imbalanced problems and threshold optimisation.

Python

```
from sklearn.metrics import roc_curve, roc_auc_score
y_prob = clf.predict_proba(X_test)[:,-1]
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
auc = roc_auc_score(y_test, y_prob)
```

QQ139 — Difference between recall and specificity?

Answer. Recall (TPR): $\frac{TP}{TP+FN}$ — proportion of actual positives captured. **Specificity (TNR):** $\frac{TN}{TN+FP}$ — proportion of actual negatives correctly identified. Use recall when missing positives is costly (medical); specificity when false positives are costly (spam).

QQ149 — What is quantisation in deploying ML models?

Answer. Quantisation reduces the numerical precision of model weights/activations (e.g. float32 to int8), making deployment efficient on edge/mobile devices with normally minor accuracy loss.

QQ156 — What is out-of-bag (OOB) error in random forests?

Answer. OOB error estimates generalisation accuracy using samples not included in each tree's bootstrap sample—a built-in cross-validation metric that eliminates the need to hold out validation data.

QQ158 — What is pipelining in MLOps?

Answer. Automating and chaining all stages of ML (data collection, preprocessing, training, testing, deployment) to improve reproducibility, monitoring, and rapid iteration (e.g. Kubeflow, MLflow, Airflow).

QQ168 — What is data sharding?

Answer. Dividing large datasets into smaller, manageable pieces (shards) stored or processed on different machines. Improves scalability and performance for distributed machine learning systems.

QQ177 — What is evolutionary computation in ML?

Answer. A family of optimisation algorithms inspired by biological evolution (e.g. genetic algorithms), used to optimise models, hyperparameters, or generate novel neural architectures.

QQ178 — What is multi-objective optimisation?

Answer. Optimisation involving more than one conflicting metric (e.g. accuracy and model size). Solutions evaluated by Pareto optimality—improvement in one objective means tradeoff in another. Example: model accuracy vs. latency trade-offs in mobile deployment.

QQ179 — What is transferability in adversarial ML?

Answer. Adversarial examples generated for one model can often fool different models (even with varied architectures/training). Implication: model robustness should be evaluated against both white-box and black-box attackers.

QQ180 — What is hyperparameter tuning with Bayesian Optimisation?

Answer. A search strategy that models the performance metric as a probabilistic function of hyperparameters (e.g. Gaussian Process), choosing new hyperparameters based on uncertainty and expected improvement. Needs fewer runs than grid/random search.

QQ195 — Why are ensemble models used in Kaggle competitions?

Answer. Ensembles combine strengths and compensate for weaknesses of various models, pushing accuracy and generalisation higher than any single model. Winning solutions often average or stack dozens of diverse models.

QQ199 — What is class imbalance and its impact?

Answer. Imbalanced data means most samples belong to one class (majority) with very few in another (minority), such as fraud or rare disease detection. Impact: models “cheat” by predicting the majority class, making minority detection poor and accuracy metrics misleading.

QQ200 — How do you address class imbalance?

Answer.

- **Rebalancing:** Oversample minority (SMOTE), undersample majority.
- **Algorithm adjustment:** Use class weights, modify decision thresholds.
- **Careful evaluation:** Focus on recall, precision, F1, ROC-AUC—never just overall accuracy.

QQ204 — What is model calibration and why is it important?

Answer. Calibration means predicted probabilities reflect real-world frequencies. Example: in a calibrated medical model, 70% risk should mean the event actually happens 70% of the time. Achieved with Platt scaling or isotonic regression.

QQ205 — What is bootstrapping and why is it used?

Answer. Bootstrapping generates many resampled datasets (with replacement) from the original data to estimate variability, build confidence intervals, or power bagging/ensembles. Provides insights into model stability and predictions.

21 Q&A — Classical Supervised Algorithms

QQ4 — Difference between classification and regression?

Answer.

Feature	Classification	Regression
Output type	Discrete categories	Continuous values
Example	Spam vs. ham	House price prediction
Algorithms	Logistic Regression, SVM	Linear Regression, SVR

QQ5 — How does k-Nearest Neighbours (kNN) work?

Answer. kNN classifies a point by looking at the k closest labelled points in feature space. For regression, it averages the values of k nearest neighbours.

Python

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
predictions = knn.predict(X_test)
```

QQ22 — What is a perceptron?

Answer. The simplest neural network—one layer with one neuron—used for binary linear classification.

Python

```
from sklearn.linear_model import Perceptron
p = Perceptron()
p.fit(X_train, y_train)
```

QQ27 — What is cross entropy loss?

Answer. Cross entropy loss measures classification model performance where output is a probability—it increases as the predicted probability diverges from the actual label.

QQ37 — What are decision trees and how do they work?

Answer. Decision trees classify data by splitting nodes on feature values, choosing splits based on Gini impurity or entropy, until reaching pure nodes or max depth.

QQ39 — What is entropy in decision trees?

Answer. Entropy measures impurity or disorder in a node, used for determining optimal splits (as in the ID3 algorithm). $H = -\sum_i p_i \log_2 p_i$.

QQ44 — Parametric vs. non-parametric models?

Answer.

Parametric	Non-Parametric
Assumes data distribution	Fewer assumptions
Fewer parameters	More flexible
Example: Linear Regression	Example: kNN, Decision Trees

QQ69 — What is the bias node in neural networks?

Answer. The bias node shifts the activation function curve and improves learning, analogous to the intercept in linear regression.

QQ71 — What is the softmax function?

Answer. Softmax converts raw scores (logits) into probabilities summing to 1—used in the output layer of multiclass classification networks.

Python

```
import numpy as np
def softmax(x):
    e_x = np.exp(x - np.max(x))
    return e_x / e_x.sum(axis=0)
```

QQ95 — What is multi-class classification?

Answer. Prediction where outputs can belong to more than two categories (e.g. classifying types of fruit).

QQ96 — What is multi-label classification?

Answer. A problem where each input can be assigned multiple labels (e.g. tagging images with multiple objects).

QQ107 — What is time series forecasting?

Answer. Predicting future values based on historical data with temporal sequence, using models like ARIMA or LSTM.

QQ108 — What is ARIMA?

Answer. Auto-Regressive Integrated Moving Average—a popular linear time series forecasting model combining autoregression, differencing, and moving average components.

QQ119 — Hard vs. soft classification?

Answer. Hard classification: Predicts class labels only. **Soft classification:** Predicts probabilities for each class.

QQ122 — What is multi-output regression?

Answer. Predicting multiple continuous values simultaneously in a single regression task.

QQ125 — What is quantile regression?

Answer. A regression technique predicting conditional quantile values (rather than the mean)—useful for modelling median or prediction intervals.

QQ126 — What is distance metric learning?

Answer. Learning an appropriate distance function (rather than using Euclidean distance) for tasks like clustering and retrieval.

QQ131 — What is ensemble stacking?

Answer. Stacking trains multiple base models, uses their out-of-fold predictions as features, then trains a meta-learner on those. At test time, base model outputs feed the meta-model for the final prediction.

Python

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
estimators = [('tree', DecisionTreeClassifier()), ('svc', SVC(probability=True))]
clf = StackingClassifier(estimators=estimators,
                        final_estimator=LogisticRegression())
clf.fit(X_train, y_train)
```

QQ154 — What is LIME?

Answer. LIME (Local Interpretable Model-agnostic Explanations) explains model predictions by locally perturbing the input and fitting a simple interpretable model on the results to approximate local behaviour.

QQ189 — What is intercept bias in regression?

Answer. The intercept (or bias) is the constant term in regression models, allowing the fitted function to shift to best match the data. Without it, predictions are forced through the origin.

22 Q&A — Ensembles & Boosting

QQ16 — Difference between bagging and boosting?

Answer.

Bagging	Boosting
Trains models in parallel	Trains models sequentially
Reduces variance	Reduces bias
Example: Random Forest	Example: AdaBoost, XGBoost

QQ32 — Name common ensemble algorithms.

Answer. Random Forest, Bagging, Boosting (AdaBoost, Gradient Boosting, XGBoost), Stacking, Voting Classifier.

QQ33 — What is stacking in ensembles?

Answer. Stacking combines predictions from several base learners (with different strengths) using a meta-learner to give the final prediction.

QQ34 — What is bagging and give an example?

Answer. Bagging fits multiple models on random data subsets with replacement and averages results. Example: Random Forest.

QQ35 — What is boosting and give an example?

Answer. Boosting trains models sequentially, each focusing on correcting previous model errors. Example: AdaBoost, Gradient Boosting.

QQ50 — How can you handle imbalanced datasets?

Answer.

- Oversample minority class (SMOTE)
- Undersample majority class
- Use class weights
- Use ensemble methods designed for imbalance

Python

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression(class_weight='balanced')
```

QQ89 — Ensemble bagging vs. boosting in one line?

Answer. Bagging averages predictions from many independent models; boosting builds models sequentially, each correcting its predecessors' errors.

23 Q&A — Unsupervised Learning & Clustering

QQ57 — What is k-means clustering?

Answer. k-means partitions data into k clusters by assigning points to nearest centroids iteratively until convergence.

Python

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3)
labels = kmeans.fit_predict(X)
```

QQ58 — What is hierarchical clustering?

Answer. A clustering method building a hierarchy either by successive merging (agglomerative) or splitting (divisive) of clusters.

QQ67 — What is anomaly detection?

Answer. Anomaly detection identifies rare items, events, or observations that differ significantly from the majority of data.

QQ70 — What is the elbow method in clustering?

Answer. A technique to determine the optimal number of clusters in k-means by plotting within-cluster sum of squares and looking for the “elbow” point where the decrease sharply slows.

QQ127 — What is the silhouette coefficient formula?

Answer. $s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$, where $a(i)$ is the average intra-cluster distance and $b(i)$ is the nearest-cluster distance. Values range from -1 to 1 (higher is better).

24 Q&A — Dimensionality Reduction

QQ14 — What is Principal Component Analysis (PCA)?

Answer. PCA is a dimensionality reduction technique transforming data into fewer uncorrelated variables (principal components), preserving as much variance as possible.

Python

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X)
```

QQ63 — Explain dimensionality reduction.

Answer. Reducing the number of input variables (e.g. via PCA, t-SNE) to speed up training, aid visualisation, and mitigate the curse of dimensionality.

QQ64 — What is t-SNE?

Answer. t-distributed Stochastic Neighbour Embedding (t-SNE) is a dimensionality reduction technique mainly for visualising high-dimensional data in 2D or 3D.

QQ175 — What is singular value decomposition (SVD)?

Answer. A factorisation method decomposing real or complex matrices into the product of three matrices. Uses: dimensionality reduction (LSA for NLP), recommender systems, image compression.

25 Q&A — Deep Learning Core Concepts

QQ24 — Explain batch gradient descent.

Answer. Batch gradient descent computes the gradient using the entire training dataset and updates parameters in each iteration—accurate but slow for large datasets.

QQ28 — What does the learning rate control?

Answer. The learning rate controls the step size during gradient descent. Too high may cause divergence; too low causes slow convergence.

QQ55 — What are recurrent neural networks (RNNs)?

Answer. RNNs are neural networks designed for sequential data (time series, text) where outputs depend on previous computations—they maintain a form of memory.

QQ72 — What is the activation function in a neural network?

Answer. An activation function introduces non-linearity, enabling the network to learn complex relationships. Examples: ReLU, sigmoid, tanh.

QQ73 — What is the ReLU activation function?

Answer. ReLU (Rectified Linear Unit) outputs zero for negative input and the input itself if positive: $f(x) = \max(0, x)$.

QQ75 — What is Xavier/Glorot initialisation?

Answer. A technique for initialising neural network weights to prevent vanishing/exploding gradients, maintaining variance through layers.

QQ77 — What is gradient vanishing/explosion?

Answer. When gradients become extremely small (vanishing) or large (exploding) during backpropagation, hindering training—especially in deep or recurrent networks.

QQ78 — What is an epoch?

Answer. One epoch means one complete pass through the entire training dataset by the model.

QQ79 — What is mini-batch gradient descent?

Answer. An optimisation algorithm updating model parameters based on a small random subset (mini-batch) per iteration—balances the speed of stochastic and stability of batch gradient descent.

QQ80 — What is the main role of the loss function?

Answer. The loss function quantifies how well predictions match actual target values, guiding weight updates during training.

QQ81 — What is a generator in deep learning?

Answer. A Python function or object yielding data batches on-the-fly—used for efficient data feeding during neural network training when the full dataset doesn't fit in memory.

QQ82 — What are Generative Adversarial Networks (GANs)?

Answer. GANs consist of a generator and discriminator network competing against each other—the generator creates fake data, the discriminator distinguishes real from fake. Training converges when the generator fools the discriminator.

QQ83 — What is LSTM?

Answer. Long Short-Term Memory is an RNN architecture designed to capture long-range dependencies and mitigate vanishing gradients, using gating mechanisms to control information flow.

QQ88 — What is a learning rate scheduler?

Answer. A tool that adjusts the learning rate during training—often reducing it when improvement plateaus—for better convergence.

QQ91 — What is the attention mechanism in deep learning?

Answer. A technique allowing neural networks to focus on specific parts of input data (such as tokens in NLP), improving performance in sequence-to-sequence tasks.

QQ101 — What is the exploding gradient problem?

Answer. When gradients grow uncontrollably, causing unstable parameter updates during training, especially in deep or recurrent architectures. Solution: gradient clipping.

QQ102 — What is a custom loss function?

Answer. A user-defined training metric implemented by subclassing or defining functions in libraries like Keras or PyTorch.

QQ128 — What mainly causes the vanishing gradient problem?

Answer. Activation functions like sigmoid and tanh squash gradients into small ranges, causing them to diminish to near-zero in deep networks during backpropagation.

QQ138 — Explain learning rate annealing and why to use it.

Answer. Learning rate annealing gradually decreases the learning rate during training, helping the model converge smoothly and escape local minima. Strategies: step decay, exponential decay, reduce on plateau.

Python

```
from keras.callbacks import ReduceLROnPlateau
ReduceLROnPlateau(monitor='val_loss', factor=0.1, patience=5)
```

QQ145 — Bag-of-words vs. embeddings for NLP?

Answer. BoW: Represents documents as word count vectors, ignoring order/context. **Embeddings:** Map words to dense vectors in a continuous space, preserving semantic relationships (e.g. king – man + woman = queen). Embeddings (Word2Vec, GloVe, BERT) are preferred for neural networks due to richer information.

QQ173 — What is a graph neural network (GNN)?

Answer. Neural networks operating directly on graphs, learning representations for nodes, edges, or whole graphs by aggregating information from a node's neighbours. Popular for: social network analysis, molecule property prediction, fraud detection.

QQ181 — What is Nash Equilibrium in ML?

Answer. In multi-agent settings, a Nash Equilibrium is where no agent can benefit by changing its own strategy if others keep theirs unchanged. Applications: multi-agent reinforcement learning, GAN training dynamics.

QQ182 — What is hierarchical reinforcement learning?

Answer. Organises learning into a hierarchy: high-level “managers” set subgoals that lower-level policies accomplish. Why: improves learning efficiency and scalability in complex, long-horizon tasks.

26 Q&A — NLP & Transformers

QQ68 — What are word embeddings?

Answer. Word embeddings are dense vector representations of words (e.g. Word2Vec, GloVe) capturing semantic meaning for use in NLP models.

QQ86 — What is Word2Vec?

Answer. An algorithm learning word embeddings by mapping words to a vector space such that similar words have close vector representations.

QQ87 — Bag of Words vs. TF-IDF?

Answer.

Bag of Words

TF-IDF

Counts word occurrences

Weights words by frequency & distinctiveness

Ignores importance

Reflects word relevance in document

QQ93 — What is BERT?

Answer. Bidirectional Encoder Representations from Transformers—a pre-trained NLP model using bidirectional transformer architecture for deep language understanding.

QQ130 — What is negative sampling?

Answer. A technique used in training word embeddings or recommendation systems, introducing fake (negative) examples to improve contrast with positive samples and speed up training.

QQ141 — What is multi-head attention in Transformers?

Answer. Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. Each “head” learns independent attention scores; outputs are concatenated. Enables learning of multiple relationships simultaneously (syntax, semantics) and boosts performance on tasks like translation and summarisation.

QQ143 — What is attention masking?

Answer. Attention masking is used in Transformers to avoid attending to certain positions, such as padding tokens or future tokens (for autoregressive tasks), ensuring the model doesn’t “cheat” by looking ahead.

27 Q&A — Reinforcement Learning & Decision Making

QQ99 — What is Q-learning?

Answer. A reinforcement learning algorithm that learns optimal actions by estimating the Q-value function (expected reward) for each state-action pair, then acting greedily.

QQ164 — What is reinforcement learning reward shaping?

Answer. Reward shaping modifies the reward function to provide more frequent or informative feedback, helping agents learn faster. Must be done carefully to avoid unintended behaviours.

QQ165 — What is Monte Carlo Tree Search?

Answer. A search strategy for decision-making combining randomness (Monte Carlo simulation) and tree search to explore promising states efficiently. At each move, simulates many games to estimate the value of each choice. Famous use: AlphaGo’s superhuman performance in Go.

QQ166 — What is bandit learning?

Answer. Multi-armed bandit problems balance “exploration” (trying new actions) and “exploitation” (choosing the best-known action) to maximise cumulative reward. Applications: online advertising, A/B testing, recommendation engines.

QQ183 — Exploration vs. exploitation in RL?

Answer. **Exploration:** Trying new actions to discover rewards. **Exploitation:** Leveraging known actions that give high rewards. Balance is key to ensuring RL agents don't get stuck in suboptimal behaviours.

28 Q&A — MLOps, Deployment & Production

QQ61 — What is model serialisation and why is it important?

Answer. Serialisation saves a trained model (e.g. using pickle or joblib) so it can be reloaded without retraining.

Python

```
import joblib
joblib.dump(model, 'model.pkl')    # Save
model = joblib.load('model.pkl')    # Load
```

QQ65 — Name tools for model deployment.

Answer. Flask, FastAPI, Docker, TensorFlow Serving, AWS Lambda, Heroku, Google Cloud ML, Azure ML.

QQ198 — Why is interpretability increasingly important in ML?

Answer. As ML models make consequential decisions (loans, diagnostics, hiring), stakeholders must understand, trust, and audit predictions. Consequences are legal, ethical, and practical—interpretability is at the heart of AI safety and responsible deployment.

29 Q&A — Explainability, Robustness & Privacy

QQ45 — What is model interpretability and why is it important?

Answer. Model interpretability means how easily a human can understand model reasoning—important for trust, debugging, and compliance.

QQ46 — List model interpretability techniques.

Answer. Feature importance, SHAP values, LIME, Partial Dependence Plots (PDPs), counterfactual explanations.

QQ104 — What is explainable AI (XAI)?

Answer. A field focused on making machine learning model predictions understandable by humans, especially for high-stakes applications.

QQ105 — What is feature importance?

Answer. A measure of the contribution of each input variable to the prediction, often provided natively by tree-based models.

QQ155 — What is SHAP and how does it differ from LIME?

Answer. SHAP (SHapley Additive exPlanations) computes feature attributions based on cooperative game theory, providing consistent, globally and locally valid explanations. LIME is local and heuristic-based; SHAP has theoretical guarantees.

QQ159 — Explainability vs. interpretability?

Answer. **Interpretability:** How well a human can understand model decisions directly (simple models, feature importance). **Explainability:** Tools/techniques to make “black box” models understandable—e.g. SHAP, LIME, counterfactual explanations.

QQ161 — What is adversarial training in deep learning?

Answer. Adversarial training makes models robust against adversarial attacks (intentionally perturbed inputs). Adversarial examples (via FGSM or PGD) are added to training data, and the model is trained to classify both clean and adversarial samples correctly. Use-case: security-sensitive applications.

QQ169 — What is federated learning?

Answer. A distributed ML approach where models are trained across multiple devices (e.g. smartphones), each with local data—without data leaving the device. Benefits: privacy, regulatory compliance, reduced centralised data storage.

QQ170 — What is privacy-preserving ML?

Answer. Methods allowing useful models to be trained or deployed without exposing sensitive data. Techniques: differential privacy, homomorphic encryption, secure multiparty computation. Use-cases: medical data, finance, federated learning.

QQ171 — What is differential privacy?

Answer. A mathematical guarantee that including or excluding a single data point minimally affects the model’s output—achieved by adding calibrated noise to data or results. Industry adoption: Apple, Google, and the U.S. Census Bureau.

QQ202 — What are SHAP values and how do they aid explainability?

Answer. SHAP values quantify each feature’s additive contribution to a model’s prediction using cooperative game theory. Strengths: local and global interpretability, theoretical consistency, applicable across many model types.

30 Q&A — Graphs & Advanced Topics

QQ150 — Explain knowledge distillation.

Answer. A technique transferring knowledge from a large, complex “teacher” model to a smaller, faster “student” model by training the student to mimic the teacher’s outputs—enabling lightweight and efficient inference.

QQ152 — What is meta-learning (“learning to learn”)?

Answer. Meta-learning algorithms learn how to adapt quickly to new tasks with minimal data by finding generalisable initialisation or design strategies—e.g. MAML (Model-Agnostic Meta-Learning).

QQ162 — What is curriculum learning?

Answer. A training strategy where the model is first exposed to easier examples, with complexity gradually increasing. Why: mimics human learning; can speed up convergence and improve generalisation in challenging deep learning tasks.

QQ172 — What is knowledge graph embedding?

Answer. A method representing entities and relationships in a knowledge graph as low-dimensional vectors. Applications: question answering, link prediction, and recommendation.

QQ174 — What is node2vec?

Answer. A scalable algorithm generating vector representations (embeddings) of nodes in large networks using biased random walks to capture diverse connectivity patterns (community and structural equivalence).

31 Q&A — Other Topics

QQ38 — What is Gini impurity?

Answer. Gini impurity measures the probability of misclassifying a randomly chosen element if assigned randomly according to class distribution in a node. $G = 1 - \sum_i p_i^2$.

QQ49 — What is class imbalance?

Answer. Class imbalance occurs when some classes are represented far more frequently than others, making model evaluation and prediction harder.

QQ94 — What is fine-tuning in deep learning?

Answer. Adapting a pre-trained model on new, task-specific data—updating some or all weights for efficient training on specific tasks without starting from scratch.

QQ97 — What is zero-shot learning?

Answer. Machine learning where the model correctly predicts on classes not seen during training, often using semantic relationships or natural language descriptions.

QQ115 — What is SMOTE?

Answer. Synthetic Minority Over-sampling Technique—generates synthetic examples for the minority class to address class imbalance.

Python

```
from imblearn.over_sampling import SMOTE
sm = SMOTE()
X_res, y_res = sm.fit_resample(X, y)
```

QQ120 — Early fusion vs. late fusion in multimodal learning?

Answer. Early fusion: Combines raw features from multiple modalities at the input level. **Late fusion:** Combines predictions or learned representations from separate modality-specific models.

QQ146 — What are residual connections and why do they work?

Answer. Residual (skip) connections add the input of a layer directly to its output: $\text{output} = F(x) + x$. Why: allows gradients to flow more easily through deep networks, eases optimisation, and enables very deep neural architectures (e.g. ResNets).

QQ184 — What is the curse of dimensionality?

Answer. As the number of features increases, data becomes exponentially sparse (more combinations to cover), distance measures lose meaning, and models require exponentially more data to generalise. Solution: dimensionality reduction, feature selection, regularisation.

QQ187 — What is continual learning?

Answer. Continual learning (lifelong learning) is the capability of models to learn new tasks sequentially while retaining knowledge from previous tasks (avoiding “catastrophic forgetting”). Importance: real-world systems (personal assistants, robots) need to adapt over time without full retraining.